

CSE108 – Computer Programming Laboratory Lab #12

Date: May 22, 2026

In this lab, you will complete the implementation of a Binary Search Tree (BST) program that reads integers from a file, inserts them into the binary search tree recursively, and analyzes search performance by counting how many recursive calls are made on average to search for all numbers in the file.

The ZIP file contains **Lab12_Code.c** and **numbers.txt** files. **Do not modify numbers.txt file.** It contains the 180 integer numbers used by the program.

The starter code (**Lab12_Code.c**) provides the complete program structure. The code is explained with detailed comments. Read and understand the code. **Your task is to implement 4 functions marked with TODO comments.**

NOTE: Only modify the four functions marked with TODO comments in Lab12_Code.c. Do not change any other code in the file.

The **TODO** comments in each function provide detailed step-by-step instructions. Follow these instructions to complete the implementations of the functions. This document also have the detailed instructions as given below.

TODO 1. Node **bst_insert(Node *root, int value)*

Recursively inserts a value into the binary search tree.

- 1) Check if root is NULL or not.
 - If it is NULL, create and return a new node using `create_node()` function.
- 2) Compare value with `root->data`.
 - If value is smaller than `root->data`, recursively insert value into left subtree.
 - If value is greater than `root->data`, recursively insert value into right subtree.
- 3) Return root pointer.

TODO 2. int *bst_search(Node *root, int target, int *recursive_call_count)*

Recursively searches for a target value in the binary search tree and counts recursive calls done for a search

- 1) Increment `recursive_call_count` variable by one since we are keeping track of the number of recursive function calls made for `bst_search()` function.
- 2) Check if root is NULL or not.
 - If it is NULL, target is not found. Return 0.
- 3) If target equals `root->data`, target has been found. Return 1.
- 4) If target is smaller than `root->data`, recursively search left subtree.
- 5) Otherwise, recursively search right subtree.

TODO 3. *Node *build_bst_helper(FILE *file, Node *root, int *count)*

This function **recursively** reads all integer numbers in the file. When an integer number is read, it must be inserted into the binary search tree.

- 1) Read one integer number from file.
 - If reading fails, return root.
- 2) Insert the newly read number into binary search tree using `bst_insert()` function.
- 3) Increase count variable by one since count variable keeps the number of nodes inside binary search tree.
- 4) Make a recursive function call to read the next number in the file.

TODO 4. *void analyze_bst_search_performance(Node *root, const char *filename, int count)*

Analyzes search performance of binary search tree

- 1) Open file with given filename for reading.
 - If file cannot be opened, print an error message and stop program with `exit(1)`.
- 2) Read all integer numbers from the file using a loop.
 - For each number:
 - Define an integer variable named `recursive_call_count`. Set the variable to 0. It will keep the number of recursive calls done for `bst_search` function.
 - Call `bst_search()` function to search for the newly read number. Make sure you pass `recursive_call_count` variable as an argument to the function properly.
 - Add `recursive_call_count` variable to the `total_recursive_calls` variable.
- 3) Close the file.

NOTE: You are given some parts of `analyze_bst_search_performance` function already. **Do not modify the parts given to you.**

Expected Output:

The number of nodes inside binary search tree: 180

===== BINARY SEARCH TREE SEARCH STATISTICS =====

For 180 numbers, each search required an average of 7.64 recursive call.